Comparison of three 20-attempt CORAL evolution runs

The runs

Run (this report's name)	CORAL run dir	Seeds available to the agent	Scoring (what the agent maximized)	Attempts (scored real / tune / crashed)	Wall time	Highest measured throughput (real)
20-all-seeds-tanh	2026-10-09_004013	All 7 reference seeds + noop start	z-score vs noop baseline, squashed with tanh, weighted with bytes read / refaults (branch master)	20 / 8 / 0	58 min	1,277.7 ops/s
20-all-seeds-abs	2026-10-11_130334	All 7 reference seeds + noop start	absolute throughput (ops/s)	19 / 0 / 1	27 min	1,262.3 ops/s
20-one-seed-abs	2026-10-09_162748	S3-FIFO is the only seed (starting file)	absolute throughput (ops/s)	19 / 8 / 2	62 min	1,276.6 ops/s

All three runs: one Claude Code agent (claude-opus-5-5), 20 real attempts as the stop condition (--tune attempts are graded identically but do not count toward the cap), the same CloudLab node and the same GET-SCAN LevelDB benchmark (4 threads, 95% GET / 5% SCAN, 64 MB page-cache cgroup, 30 s measured phase). Throughput in this report is always the raw ops/s the benchmark measured, even for the tanh run whose own score was a squashed z-score.

Headline

• **Highest measured throughput of any real attempt: 1,277.7 ops/s** — **20-all-seeds-tanh**, attempt #11 (2d5d649a), an evolved S3-FIFO variant (+2.6% vs the unmodified S3-FIFO mean of 1,245.2 ops/s, n=5). Its full code follows the comparison tables, then the best evolved policy of each of the other two runs.

• The three runs' best real throughputs are 20-all-seeds-tanh 1,277.7, 20-all-seeds-abs 1,262.3, 20-one-seed-abs 1,276.6 ops/s — within about 1.2% of each other.

• **Noise is as large as the differences.** Unmodified S3-FIFO alone measured 1,216.7 … 1,263.0 ops/s (3.7% of its mean) across 5 runs/attempts, and identical code re-run inside a run moved by up to several percent (see the noise table). No run's best is clearly separated from the unmodified S3-FIFO or from the other runs' bests.

• There is no evidence here that seeding all seven policies beat seeding S3-FIFO alone, or that the tanh/bytes-weighted score beat raw throughput: every run's best is a small modification of S3-FIFO (or, in 20-all-seeds-abs, an unmodified MGLRU measurement).

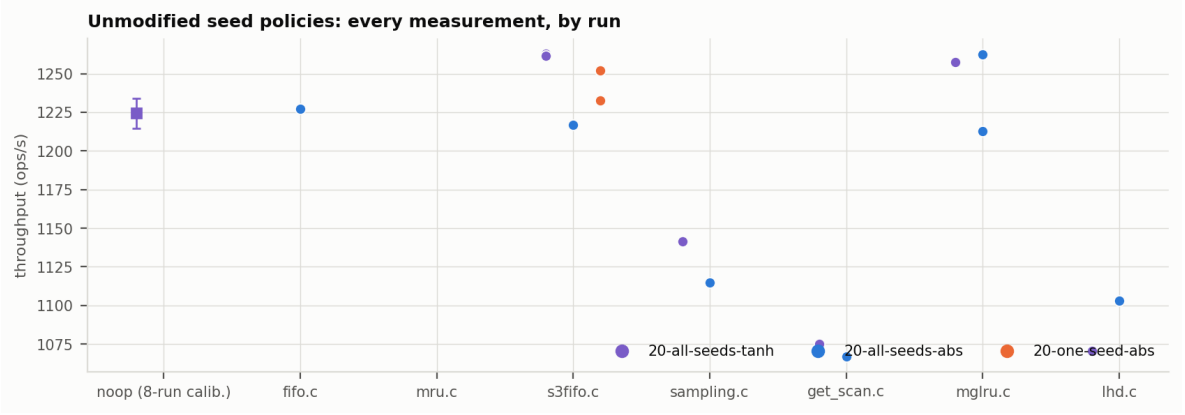
Seed policies

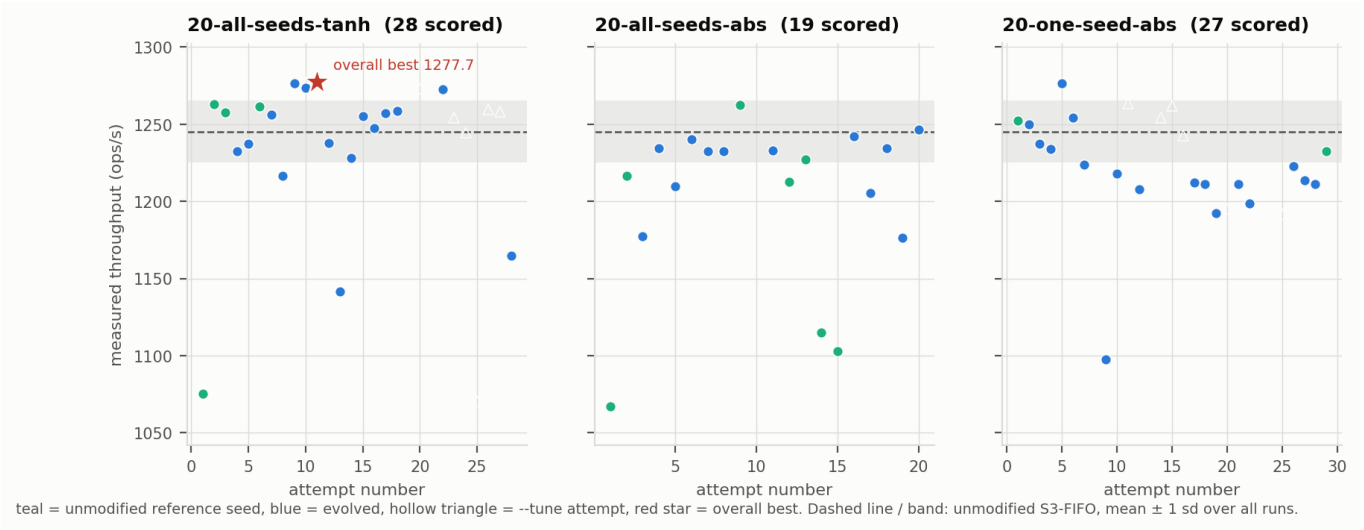
Every run starts from `seed/noop.c` (the no-op / kernel-default-reclaim policy, except 20-one-seed-abs whose starting file is S3-FIFO). The two all-seeds runs also exposed seven complete reference policies in `reference/` that the agent could copy or adapt; 20-one-seed-abs exposed none. Measurements are single evals of the unmodified file.

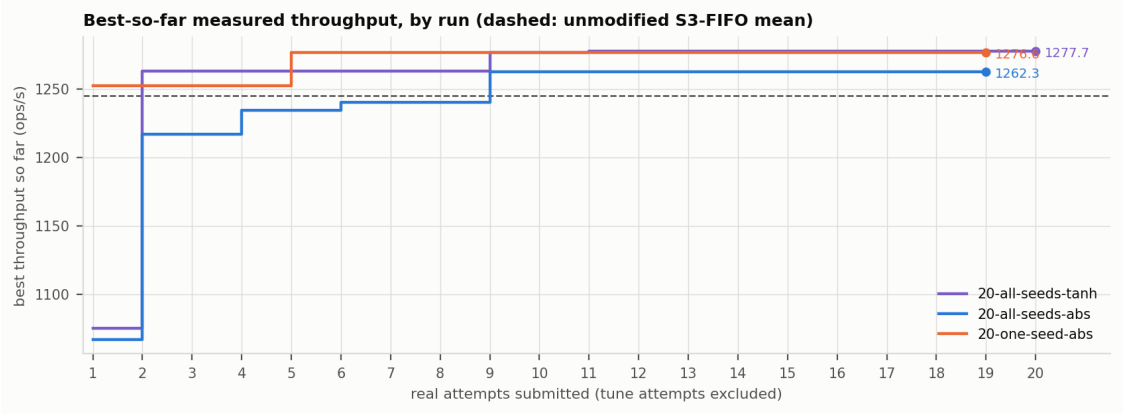
Seed	What it does	tanh run: measured (ops/s)	all-seeds-abs: measured	one-seed-abs: measured
noop.c	No-op: registers struct_ops but makes no eviction decisions; vanilla kernel reclaim, the reference starting point.	calibration: 1,224.3 ± 9.5 (n=8, not attempts)	not evaluated	not available (S3-FIFO is the start)
fifo.c	FIFO: a single list. New folios are appended at the tail and eviction walks from the head, so the oldest-inserted folio goes first. Accesses are ignored entirely (no folio_accessed hook).	not run	#13: 1,227.3	not available
mru.c	MRU: a single list. Added and accessed folios move to the head, and eviction walks from the head, so the MOST recently used folio is evicted first. Suited to cyclic/sequential scans larger than the cache; poor for skewed reuse.	not run	not run	not available
s3fifo.c	S3-FIFO: a small FIFO (~10% of the cache) admits new folios; a folio re-accessed while in the small queue is promoted to the main queue, otherwise evicted. The main queue is CLOCK-like with a frequency counter capped at 3. A ghost map remembers recently evicted folios so a quick re-reference is admitted straight into main.	#2: 1,263.0, #6: 1,261.5	#2: 1,216.7	#1: 1,252.3, #29: 1,232.5
sampling.c	Sampled LFU: one list plus a per-folio access counter. To evict, sample 20 candidates from the list and take those with the lowest counts. LevelDB heuristic: the last page of each file (the index block) gets +100000 so it is effectively never evicted.	#19: 1,141.3	#14: 1,114.8	not available
get_scan.c	GET-SCAN policy (written for this workload): two lists, one for folios added by the scan thread (recognised through the harness-pinned scan_pids map) and one for everything else. Eviction does sampled LFU (5 candidates, LevelDB index-block bonus), taking from the scan list once it holds >= 5000 pages and from the general list otherwise; with no scan pages tracked it proposes no victims.	#1: 1,075.1	#1: 1,067.0	not available
mglru.c	MGLRU: a BPF port of Linux's multi-generational LRU. Folios sit in generations (by age) and tiers (by access frequency); aging advances the generation counters and eviction takes from the oldest generation, with a ghost map feeding refault information back.	#3: 1,257.6	#9: 1,262.3, #12: 1,212.8	not available
lhd.c	LHD (Least Hit Density): ranks folios by expected hits per unit of cache space-time, using per-age hit/eviction statistics, and evicts the lowest density first. Class statistics are recomputed periodically: a BPF event wakes the userspace loader through a ring buffer and it runs the BPF 'reconfigure' program.	#25: 1,070.5	#15: 1,103.1	not available

Unmodified S3-FIFO is the only seed measured in all three runs: 20-all-seeds-tanh 1,263.0, 20-all-seeds-tanh 1,261.5, 20-all-seeds-abs 1,216.7, 20-one-seed-abs 1,252.3, 20-one-seed-abs 1,232.5 ops/s (mean 1,245.2, sd 20.0, n=5). The agents ran the other reference policies unmodified only as baselines before adapting them.

Seeds, every attempt, and best-so-far







Best policy of each run

Run	Selection	Attempt	Class	Policy / change	Throughput (ops/s)	vs unmodified S3-FIFO mean
20-all-seeds-tanh	run's own score (saved as best/)	#27 50f29a63	tune	repeat #2 of freq-cap-2 gated-ghost s3fifo (e573de75: 1259.7 ops/s, 308.4 MB) to compare mean vs b631bf1b (3 runs: mean 1268 ops/s, 309.0 MB)	1,258.36	+1.06%
20-all-seeds-tanh	highest measured throughput, real attempts	#11 2d5d649a	real	gated-ghost s3fifo (ghost 4x) with small queue 1/8 of cache (bigger small queue cut I/O in eval 7; throughput has slack now)	1,277.66	+2.61%
20-all-seeds-tanh	highest measured throughput, evolved (non-seed) real	#11 2d5d649a	real	gated-ghost s3fifo (ghost 4x) with small queue 1/8 of cache (bigger small queue cut I/O in eval 7; throughput has slack now)	1,277.66	+2.61%
20-all-seeds-abs	run's own score (saved as best/)	#9 3233ed4a	real	mglru.c unmodified — baseline: reference mglru.c unchanged (family comparison vs s3fifo ~1217)	1,262.31	+1.37%
20-all-seeds-abs	highest measured throughput, real attempts	#9 3233ed4a	real	mglru.c unmodified — baseline: reference mglru.c unchanged (family comparison vs s3fifo ~1217)	1,262.31	+1.37%
20-all-seeds-abs	highest measured throughput, evolved (non-seed) real	#20 7f1ae77b	real	new-page grace: s3fifo small-FIFO share cache/15 -> cache/6 (opposite of d7fd5641's /40 which lost 5%; 4 runs show evicting new pages fast hurts)	1,246.43	+0.10%
20-one-seed-abs	run's own score (saved as best/)	#24 260db443	tune	INSTRUMENTATION v6: why eviction requests go unfilled (got~req/2): count iterate outcomes (scan/small notready, evict, promote) + short calls. trunc t	1,276.89	+2.55%
20-one-seed-abs	highest measured throughput, real attempts	#5 30ecfc5d	real	LFU-with-history: ghost map stores u32 access count (4x cache entries), restored on ghost-hit readmission; main evicted by sampling 32 and taking lowe	1,276.64	+2.53%
20-one-seed-abs	highest measured throughput, evolved (non-seed) real	#5 30ecfc5d	real	LFU-with-history: ghost map stores u32 access count (4x cache entries), restored on ghost-hit readmission; main evicted by sampling 32 and taking lowe	1,276.64	+2.53%

Shaded: the overall best (highest measured throughput among all real attempts in the three runs). The tanh run's own score ranked a different attempt first because that score rewarded fewer bytes read and refaults as well as throughput; the comparison here is by measured throughput only.

Identical code measured more than once (noise)

Run	Code (attempts)	Measurements (ops/s)	Range % of mean
20-all-seeds-tanh	s3fifo.c unmodified (#2, #6)	1,263.0, 1,261.5	0.1
20-all-seeds-tanh	gated-ghost s3fifo with ghost 4x cache (only GET-thread re-add (#10, #17, #20)	1,273.7, 1,257.4, 1,273.1	1.3
20-all-seeds-tanh	gated-ghost s3fifo (61054f8b) with freq cap 2 instead of 3: fe (#26, #27, #28)	1,259.7, 1,258.4, 1,165.0	7.7
20-all-seeds-abs	s3fifo+scan list: raise scan-list floor 64 -> cache_size/16 so (#4, #6)	1,234.5, 1,240.1	0.4
20-all-seeds-abs	mglru.c unmodified (#9, #12)	1,262.3, 1,212.8	4.0
20-all-seeds-abs	scan-favoring 1/3: invert priority - evict GET pages (s3fifo s (#16, #18)	1,242.2, 1,234.4	0.6
20-one-seed-abs	s3fifo.c unmodified (#1, #29)	1,252.3, 1,232.5	1.6
20-one-seed-abs	LFU-with-history: ghost map stores u32 access count (4x cache (#5, #7)	1,276.6, 1,223.6	4.2
20-one-seed-abs	LFU-hist + 16x history (eval6 cfg) + SCAN_MIN 512->128 pages ((#10, #11)	1,217.9, 1,263.4	3.7

The overall best (#11 in 20-all-seeds-tanh) was measured once. Its closest relative in the same run, b631bf1b (the same design with a 1/15 small queue), measured 1273.7, 1257.4 and 1273.1 ops/s — a mean near 1268 — so the true level of this design is probably a little below its single 1277.7 reading.

The best policy: 20-all-seeds-tanh attempt #11 (2d5d649a), 1,277.66 ops/s

What it is: gated-ghost s3fifo (ghost 4x) with small queue 1/8 of cache (bigger small queue cut I/O in eval 7; throughput has slack now). In practice it is the S3-FIFO reference policy with four small changes (diff below), made by the agent in the 20-all-seeds-tanh run (its commit message says a bigger small queue had cut I/O in an earlier eval):

1. Small queue 1/15 → 1/8 of the cache (`cache_size / 15` → `cache_size / 8`): new folios are kept in the small FIFO longer before eviction decisions move them, so more one-touch pages are filtered in the small queue before reaching the main queue.

2. Scan awareness: reuses the harness-pinned `scan_pids` map (userspace loader reopens `/sys/fs/bpf/cache_ext/scan_pids`) and adds `is_scanning_pid()`.

3. Gated ghost admission (`if (!is_scanning_pid() && folio_in_ghost(folio))`): a re-read of a recently evicted page by the scan thread no longer jumps straight into the main queue; only GET-thread reuse earns that.

4. Ghost history 4× larger (`ghost_map max_entries = 4 × cache_size`): remembers evictions for longer, so the Zipf-hot GET keys are recognised on return.

Source file: `results/best/run3_best_throughput_2d5d649a.c` (and `noop.c` at that commit in the run's agent worktree). Highlighted lines in the listing below are the lines that differ from the S3-FIFO reference.

Full source

```
// ================================================================================
// SECTION: BPF KERNEL CODE
// ================================================================================
// EVOLVE-BLOCK-START
// S3-FIFO: a small FIFO (~10% of the cache) admits new folios; a folio re-accessed while in
// the small queue is promoted to the main queue, otherwise evicted. The main queue is
// CLOCK-like with a frequency counter capped at 3. A ghost map remembers recently evicted
// folios so a quick re-reference is admitted straight into main.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ENOENT        2  /* include/uapi/asm-generic/errno-base.h */
#define INT64_MAX    (9223372036854775807LL)

// Set from userspace. In terms of number of pages.
// TODO: change

//#define CACHE_SIZE (((1ull << 30) * 2) / 4096)
#define CACHE_SIZE (((1ull << 20) * 200) / 4096)
const volatile size_t cache_size = 0;

struct folio_metadata {
    s64 freq;
    bool in_main;
};

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, 4000000);
} folio_metadata_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    //__uint(max_entries, CACHE_SIZE); // TODO: change
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

/* Harness-pinned map of scan-thread tids (reused by the loader). */
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, int);
    __type(value, bool);
    __uint(max_entries, 100);
} scan_pids SEC(".maps");

static inline bool is_scanning_pid(void) {
    int pid = (int)(bpf_get_current_pid_tgid() & 0xFFFFFFFF);
    return bpf_map_lookup_elem(&scan_pids, &pid) != NULL;
}

static u64 main_list;
static u64 small_list;

/*
 * This is an approximate value based on what we choose to evict, not what is
 * actually evicted.
 */
static s64 small_list_size = 0;
static s64 main_list_size = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline struct folio_metadata *get_folio_metadata(struct folio *folio) {
    u64 key = (u64)folio;
    return bpf_map_lookup_elem(&folio_metadata_map, &key);
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline bool folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    // TODO: handle non-ENOENT errors
    return bpf_map_delete_elem(&ghost_map, &key) != -ENOENT;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created main_list: %llu\n", main_list);

    small_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (small_list == 0) {
        bpf_printk("cache_ext: init: Failed to create small_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created small_list: %llu\n", small_list);

    return 0;
}

static s64 bpf_s3fifo_score_main_fn(struct cache_ext_list_node *a) {
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return INT64_MAX;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return INT64_MAX;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return INT64_MAX;
    }

    s64 freq = __sync_sub_and_fetch(&data->freq, 1);
```

```
    if (freq < 0)
        data->freq = 0;

    return freq;
}

static int bpf_s3fifo_score_small_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Move to main list if freq > 1
    if (data->freq > 1) {
        data->in_main = true;
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Else, evict
    return CACHE_EXT_EVICT_NODE;
}

static void evict_main(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct sampling_options opts = {
        .sample_size = 10,
    };

    if (bpf_cache_ext_list_sample(memcg, main_list, bpf_s3fifo_score_main_fn, &opts,
                      eviction_ctx)) {
        bpf_printk("cache_ext: evict: Failed to sample main_list\n");
        return;
    }

    // if (__sync_sub_and_fetch(&main_list_size, eviction_ctx->nr_folios_to_evict) < 0)
    //     main_list_size = 0;
}

#define MAIN_ITER_FN(id)                                    \
static int bpf_s3fifo_score_main_iter_fn_##id(int idx, struct cache_ext_list_node *a)     \
{                                                           \
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))         \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                                            \
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))         \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                                            \
    struct folio_metadata *data = get_folio_metadata(a->folio);             \
    if (!data) {                                            \
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");         \
        return CACHE_EXT_CONTINUE_ITER;                     \
    }                                                        \
                                                            \
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);                \
    if (freq < id) {                                        \
        /*data->freq = 0;*/                                 \
        return CACHE_EXT_EVICT_NODE;                         \
    }                                                        \
                                                            \
    return CACHE_EXT_CONTINUE_ITER;                         \
}

MAIN_ITER_FN(0)
MAIN_ITER_FN(1)
MAIN_ITER_FN(2)
MAIN_ITER_FN(3)

static void evict_main_iter(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_0, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_1, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_2, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_3, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    }
}

static void evict_small(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 1, move to main list, otherwise evict.
     * (When evicting, move to tail in the meantime).
     *
     * Use the iterate interface.
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, small_list, bpf_s3fifo_score_small_fn, &opts,
                            eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate small_list\n");
        return;
    }

    if (__sync_fetch_and_sub(&small_list_size, opts.nr_folios_continue) < 0)
        small_list_size = 0;

    if (__sync_fetch_and_add(&main_list_size, opts.nr_folios_continue) < 0)
        main_list_size = opts.nr_folios_continue;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    // bpf_printk("cache_ext: evict_folios: main_list_size: %lld, small_list_size: %lld, cache_size: %lld\n",
    //         main_list_size, small_list_size, cache_size);
    if (small_list_size >= cache_size / 8 || main_list_size <= 2 * small_list_size)
        evict_small(eviction_ctx, memcg);
    else
        evict_main_iter(eviction_ctx, memcg);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        bpf_printk("cache_ext: accessed: Failed to get metadata\n");
        return;
    }
```

```
314
315     // Cap frequency at 3
316     if (__sync_add_and_fetch(&data->freq, 1) > 3)
317         data->freq = 3;
318 }
319
320 void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
321     u64 key = (u64)folio;
322     u8 ghost_val = 0;
323
324     // if (bpf_cache_ext_list_del(folio)) {
325     //     bpf_printk("cache_ext: Failed to delete folio from sampling_list\n");
326     //     return;
327     // }
328
329     struct ghost_entry ghost_key = {
330         .address_space = (u64)folio->mapping->host,
331         .offset = folio->index,
332     };
333
334     // Don't return early, we want to delete the folio metadata regardless
335     if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
336         bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");
337
338     struct folio_metadata *data = get_folio_metadata(folio);
339     if (!data) {
340         //bpf_printk("cache_ext: evicted: Failed to get metadata\n");
341         return;
342     }
343
344     if (data->in_main)
345         __sync_fetch_and_sub(&main_list_size, 1);
346     else
347         __sync_fetch_and_sub(&small_list_size, 1);
348
349     bpf_map_delete_elem(&folio_metadata_map, &key);
350
351     // if (bpf_map_delete_elem(&folio_metadata_map, &key))
352     //     bpf_printk("cache_ext: evicted: Failed to delete metadata\n");
353 }
354
355 /*
356  * If folio is in the ghost map, add to tail of main list, otherwise add to tail
357  * of small list.
358  */
359 void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
360     if (!is_folio_relevant(folio))
361         return;
362
363     u64 key = (u64)folio;
364     struct folio_metadata new_meta = {
365         .freq = 0,
366     };
367
368     u64 list_to_add;
369     // Scan re-reads of ghosted pages stay in small: only GET reuse earns main directly.
370     if (!is_scanning_pid() && folio_in_ghost(folio)) {
371         list_to_add = main_list;
372         new_meta.in_main = true;
373         __sync_fetch_and_add(&main_list_size, 1);
374     } else {
375         list_to_add = small_list;
376         new_meta.in_main = false;
377         __sync_fetch_and_add(&small_list_size, 1);
378     }
379
380     if (bpf_cache_ext_list_add_tail(list_to_add, folio)) {
381         // TODO: add back to ghost_map?
382         bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
383         return;
384     }
385
386     if (bpf_map_update_elem(&folio_metadata_map, &key, &new_meta, BPF_ANY)) {
387         // TODO: add back to ghost_map? + error check delete call?
388         bpf_cache_ext_list_del(folio);
389         bpf_printk("cache_ext: added: Failed to create folio metadata\n");
390         return;
391     }
392 }
393
394 SEC(".struct_ops.link")
395 struct cache_ext_ops evo_policy_ops = {
396     .init = (void *)evo_policy_init,
397     .evict_folios = (void *)evo_policy_evict_folios,
398     .folio_accessed = (void *)evo_policy_folio_accessed,
399     .folio_evicted = (void *)evo_policy_folio_evicted,
400     .folio_added = (void *)evo_policy_folio_added,
401 };
402 // EVOLVE-BLOCK-END
403
404 // ============================================================================
405 // SECTION: USERSPACE LOADER
406 // ============================================================================
407 // Loader for the S3-FIFO policy: parses -w/-s/-c like every policy in this repo,
408 // attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
409 // EVOLVE-BLOCK-START
410 #include <argp.h>
411 #include <errno.h>
412 #include <stdlib.h>
413 #include <bpf/bpf.h>
414 #include <fcntl.h>
415 #include <limits.h>
416 #include <signal.h>
417 #include <stdint.h>
418 #include <stdio.h>
419 #include <string.h>
420 #include <sys/stat.h>
421 #include <sys/types.h>
422 #include <unistd.h>
423
424 #include "dir_watcher.h"
425 #include "evo_policy.skel.h"
426
427 char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_size <size> --cgroup_path <path>\n";
428 struct cmdline_args {
429     char *watch_dir;
430         uint64_t cgroup_size;
431         char *cgroup_path;
432 };
433
434 static struct argp_option options[] = {
435     { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
436         {"cgroup_size", 's', "SIZE", 0, "Size of the cgroup"},
437         {"cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)"},
438     { 0 },
439 };
440
441 static const uint64_t page_size = 4096;
442
443 static volatile sig_atomic_t exiting;
444
445 static void sig_handler(int signo) {
446     exiting = 1;
447 }
448
449 static error_t parse_opt(int key, char *arg, struct argp_state *state)
450 {
451     struct cmdline_args *args = state->input;
452     switch (key) {
453     case 'w':
454         args->watch_dir = arg;
455         break;
456         case 's':
457                 // TODO: move this to parse_args()
458                 errno = 0;
459                 args->cgroup_size = strtoull(arg, NULL, 10);
460                 if (errno)
461                         args->cgroup_size = 0;
462
463                 break;
464         case 'c':
465                 args->cgroup_path = arg;
466                 break;
467     default:
468         return ARGP_ERR_UNKNOWN;
469     }
470     return 0;
471 }
472
473 static int parse_args(int argc, char **argv, struct cmdline_args *args) {
474     struct argp argp = { options, parse_opt, 0, 0 };
475     argp_parse(&argp, argc, argv, 0, 0, args);
476
477     if (args->watch_dir == NULL) {
478         fprintf(stderr, "Missing required argument: watch_dir\n");
479         return 1;
480     }
481
482     if (args->cgroup_size == 0) {
483             fprintf(stderr, "Invalid cgroup size\n");
484             return 1;
485     }
486
487     if (args->cgroup_path == NULL) {
488         fprintf(stderr, "Missing required argument: cgroup_path\n");
489         return 1;
490     }
491
```

```
    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set cache size in terms of number of pages. Assumes uniform page size.
    skel->rodata->cache_size = args.cgroup_size / page_size;
    fprintf(stderr, "Cgroup size: %lu bytes\n", args.cgroup_size);
    fprintf(stderr, "Cache size: %lu pages\n", skel->rodata->cache_size);

    // Resize ghost_map
    if (bpf_map__set_max_entries(skel->maps.ghost_map, 4 * skel->rodata->cache_size)) {
        perror("Failed to resize ghost_map");
        ret = 1;
        goto cleanup;
    }

    // Set watch_dir
    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    // The harness pins scan_pids before any policy loads; reuse it.
    {
        int scan_pids_fd = bpf_obj_get("/sys/fs/bpf/cache_ext/scan_pids");
        if (scan_pids_fd >= 0) {
            if (bpf_map__reuse_fd(skel->maps.scan_pids, scan_pids_fd))
                fprintf(stderr, "Failed to reuse scan_pids map fd\n");
            close(scan_pids_fd);
        } else {
            fprintf(stderr, "no scan_pids pin found\n");
        }
    }

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        ret = 1;
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        ret = 1;
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    printf("evo_policy (S3-FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

Best evolved policy of 20-all-seeds-abs: attempt #20 (7f1ae77b), 1,246.43 ops/s

What it is: new-page grace: s3fifo small-FIFO share cache/15 -> cache/6 (opposite of d7fd5641's /40 which lost 5%; 4 runs show evicting new pages fast hurts). Its 700-line source differs from the S3-FIFO reference (`seed/reference/s3fifo.c`) in 105 lines (highlighted below). It is the highest-throughput real attempt in this run that is not an unmodified seed (+0.10% vs the unmodified S3-FIFO mean).

Source file: `best/best_policy.c` in the run's report folder holds this run's own-score best; this attempt's file is `noop.c` at that commit in the run's agent worktree.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// S3-FIFO + scan list: scan-thread inserts go to a FIFO scan list evicted first. S3-FIFO: a small FIFO (~10% of the cache) admits new folios; a folio re-accessed while in
// the small queue is promoted to the main queue, otherwise evicted. The main queue is
// CLOCK-like with a frequency counter capped at 3. A ghost map remembers recently evicted
// folios so a quick re-reference is admitted straight into main.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ENOENT        2  /* include/uapi/asm-generic/errno-base.h */
#define INT64_MAX    (9223372036854775807LL)

// Set from userspace. In terms of number of pages.
// TODO: change

//#define CACHE_SIZE (((1ull << 30) * 2) / 4096)
#define CACHE_SIZE (((1ull << 20) * 200) / 4096)
const volatile size_t cache_size = 0;

struct folio_metadata {
    s64 freq;
    bool in_main;
    bool in_scan;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, int);
    __type(value, bool);
    __uint(max_entries, 100);
} scan_pids SEC(".maps");

static inline bool is_scanning_pid(void) {
    int pid = (int)(bpf_get_current_pid_tgid() & 0xFFFFFFFF);
    return bpf_map_lookup_elem(&scan_pids, &pid) != NULL;
}

static u64 scan_list;
static s64 scan_list_size = 0;
#define SCAN_MIN (cache_size / 16)
#define GET_FLOOR (cache_size / 4)

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, 4000000);
} folio_metadata_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    //__uint(max_entries, CACHE_SIZE); // TODO: change
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

static u64 main_list;
static u64 small_list;

/*
 * This is an approximate value based on what we choose to evict, not what is
 * actually evicted.
 */
static s64 small_list_size = 0;
static s64 main_list_size = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline struct folio_metadata *get_folio_metadata(struct folio *folio) {
    u64 key = (u64)folio;
    return bpf_map_lookup_elem(&folio_metadata_map, &key);
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline bool folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    // TODO: handle non-ENOENT errors
    return bpf_map_delete_elem(&ghost_map, &key) != -ENOENT;
}

static inline bool is_last_page_in_file(struct folio *folio)
{
    struct inode *inode = folio->mapping->host;
    unsigned long long file_size = i_size_read(inode);
    if (file_size == 0)
        return false;
    return folio_index(folio) == ((file_size + 4095) >> 12) - 1;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created main_list: %llu\n", main_list);

    small_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (small_list == 0) {
        bpf_printk("cache_ext: init: Failed to create small_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created small_list: %llu\n", small_list);

    scan_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (scan_list == 0)
        return -1;

    return 0;
}

static s64 bpf_s3fifo_score_main_fn(struct cache_ext_list_node *a) {
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return INT64_MAX;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return INT64_MAX;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return INT64_MAX;
    }

    s64 freq = __sync_sub_and_fetch(&data->freq, 1);
    if (freq < 0)
        data->freq = 0;

    return freq;
}

```

```
static int bpf_s3fifo_score_small_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Move to main list if freq > 1
    if (data->freq > 1) {
        data->in_main = true;
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Else, evict
    return CACHE_EXT_EVICT_NODE;
}

static int bpf_scan_evict_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (data && data->freq > 0) {
        // re-used by a GET thread: promote to main
        data->in_main = true;
        data->in_scan = false;
        return CACHE_EXT_CONTINUE_ITER;
    }
    return CACHE_EXT_EVICT_NODE;
}

static void evict_scan(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };
    if (bpf_cache_ext_list_iterate_extended(memcg, scan_list, bpf_scan_evict_fn, &opts,
                        eviction_ctx) < 0)
        return;
    if (__sync_fetch_and_sub(&scan_list_size, opts.nr_folios_continue) < 0)
        scan_list_size = 0;
    __sync_fetch_and_add(&main_list_size, opts.nr_folios_continue);
}

static void evict_main(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct sampling_options opts = {
        .sample_size = 10,
    };

    if (bpf_cache_ext_list_sample(memcg, main_list, bpf_s3fifo_score_main_fn, &opts,
                      eviction_ctx)) {
        bpf_printk("cache_ext: evict: Failed to sample main_list\n");
        return;
    }

    // if (__sync_sub_and_fetch(&main_list_size, eviction_ctx->nr_folios_to_evict) < 0)
    //     main_list_size = 0;
}

#define MAIN_ITER_FN(id)                                \
static int bpf_s3fifo_score_main_iter_fn_##id(int idx, struct cache_ext_list_node *a)     \
{                                                       \
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))         \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                                        \
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))         \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                                        \
    struct folio_metadata *data = get_folio_metadata(a->folio);             \
    if (!data) {                                        \
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");         \
        return CACHE_EXT_CONTINUE_ITER;                     \
    }                                                   \
                                                        \
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);                \
    if (freq < id) {                                    \
        /*data->freq = 0;*/                             \
        return CACHE_EXT_EVICT_NODE;                        \
    }                                                   \
                                                        \
    return CACHE_EXT_CONTINUE_ITER;                     \
}

MAIN_ITER_FN(0)
MAIN_ITER_FN(1)
MAIN_ITER_FN(2)
MAIN_ITER_FN(3)

static void evict_main_iter(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_0, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_1, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_2, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_3, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    }
}

static void evict_small(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 1, move to main list, otherwise evict.
     * (When evicting, move to tail in the meantime).
     *
     * Use the iterate interface.
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, small_list, bpf_s3fifo_score_small_fn, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate small_list\n");
        return;
    }

    if (__sync_fetch_and_sub(&small_list_size, opts.nr_folios_continue) < 0)
```

```
        small_list_size = 0;

    if (__sync_fetch_and_add(&main_list_size, opts.nr_folios_continue) < 0)
        main_list_size = opts.nr_folios_continue;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    // bpf_printk("cache_ext: evict_folios: main_list_size: %lld, small_list_size: %lld, cache_size: %lld\n",
    //        main_list_size, small_list_size, cache_size);
    // Scan-favoring: the single scan thread appears to bound total throughput, so
    // evict GET pages first and only touch the scan list when GET pages run low.
    if (small_list_size + main_list_size > GET_FLOOR) {
        if (small_list_size >= cache_size / 6 || main_list_size <= 2 * small_list_size)
            evict_small(eviction_ctx, memcg);
        else
            evict_main_iter(eviction_ctx, memcg);
        if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
            return;
    }
    evict_scan(eviction_ctx, memcg);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        bpf_printk("cache_ext: accessed: Failed to get metadata\n");
        return;
    }

    // Scan-thread touches are not reuse signals.
    if (data->in_scan && is_scanning_pid())
        return;

    // Cap frequency at 3
    if (__sync_add_and_fetch(&data->freq, 1) > 3)
        data->freq = 3;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
    u64 key = (u64)folio;
    u8 ghost_val = 0;

    // if (bpf_cache_ext_list_del(folio)) {
    //     bpf_printk("cache_ext: Failed to delete folio from sampling_list\n");
    //     return;
    // }

    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };

    // Don't return early, we want to delete the folio metadata regardless
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
        bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        //bpf_printk("cache_ext: evicted: Failed to get metadata\n");
        return;
    }

    if (data->in_scan)
        __sync_fetch_and_sub(&scan_list_size, 1);
    else if (data->in_main)
        __sync_fetch_and_sub(&main_list_size, 1);
    else
        __sync_fetch_and_sub(&small_list_size, 1);

    bpf_map_delete_elem(&folio_metadata_map, &key);

    // if (bpf_map_delete_elem(&folio_metadata_map, &key))
    //     bpf_printk("cache_ext: evicted: Failed to delete metadata\n");
}

/*
 * If folio is in the ghost map, add to tail of main list, otherwise add to tail
 * of small list.
 */
void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    u64 key = (u64)folio;
    struct folio_metadata new_meta = {
        .freq = 0,
    };

    u64 list_to_add;
    new_meta.in_scan = false;
    if (is_scanning_pid()) {
        list_to_add = scan_list;
        new_meta.in_scan = true;
        __sync_fetch_and_add(&scan_list_size, 1);
    } else if (is_last_page_in_file(folio)) {
        // LevelDB index block: needed by every GET on this table
        list_to_add = main_list;
        new_meta.in_main = true;
        new_meta.freq = 3;
        __sync_fetch_and_add(&main_list_size, 1);
    } else if (folio_in_ghost(folio)) {
        list_to_add = main_list;
        new_meta.in_main = true;
        __sync_fetch_and_add(&main_list_size, 1);
    } else {
        list_to_add = small_list;
        new_meta.in_main = false;
        __sync_fetch_and_add(&small_list_size, 1);
    }

    if (bpf_cache_ext_list_add_tail(list_to_add, folio)) {
        // TODO: add back to ghost_map?
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }

    if (bpf_map_update_elem(&folio_metadata_map, &key, &new_meta, BPF_ANY)) {
        // TODO: add back to ghost_map? + error check delete call?
        bpf_cache_ext_list_del(folio);
        bpf_printk("cache_ext: added: Failed to create folio metadata\n");
        return;
    }
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the S3-FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_size <size> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
        uint64_t cgroup_size;
        char *cgroup_path;
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
        {"cgroup_size", 's', "SIZE", 0, "Size of the cgroup"},
        {"cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)"},
    { 0 },
};

static const uint64_t page_size = 4096;

```

```c
static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
        case 's':
                // TODO: move this to parse_args()
                errno = 0;
                args->cgroup_size = strtoull(arg, NULL, 10);
                if (errno)
                        args->cgroup_size = 0;

                break;
        case 'c':
                args->cgroup_path = arg;
                break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_size == 0) {
             fprintf(stderr, "Invalid cgroup size\n");
             return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set cache size in terms of number of pages. Assumes uniform page size.
    skel->rodata->cache_size = args.cgroup_size / page_size;
    fprintf(stderr, "Cgroup size: %lu bytes\n", args.cgroup_size);
    fprintf(stderr, "Cache size: %lu pages\n", skel->rodata->cache_size);

    // Resize ghost_map
    if (bpf_map__set_max_entries(skel->maps.ghost_map, skel->rodata->cache_size)) {
        perror("Failed to resize ghost_map");
        ret = 1;
        goto cleanup;
    }

    // Set watch_dir
    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    {
        int scan_pids_fd = bpf_obj_get("/sys/fs/bpf/cache_ext/scan_pids");
        if (scan_pids_fd >= 0) {
            if (bpf_map__reuse_fd(skel->maps.scan_pids, scan_pids_fd))
                fprintf(stderr, "Failed to reuse scan_pids map fd\n");
            close(scan_pids_fd);
        } else {
            fprintf(stderr, "no scan_pids pin found\n");
        }
    }

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        ret = 1;
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        ret = 1;
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    printf("evo_policy (S3-FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

```

```
694 cleanup:
695     close(cgroup_fd);
696     bpf_link__destroy(link);
697     evo_policy_bpf__destroy(skel);
698     return ret;
699 }
700 // EVOLVE-BLOCK-END
```

Best evolved policy of 20-one-seed-abs: attempt #5 (30ecfc5d), 1,276.64 ops/s

What it is: LFU-with-history: ghost map stores u32 access count (4x cache entries), restored on ghost-hit readmission; main evicted by sampling 32 and taking lowest count (no decrement/aging); uncapped freq. Small FIFO + scan list unchanged.. Its 699-line source differs from the S3-FIFO reference (seed/`reference/s3fifo.c`) in 118 lines (highlighted below). It is the highest-throughput real attempt in this run that is not an unmodified seed (+2.53% vs the unmodified S3-FIFO mean).

Source file: `best/best_policy.c` in the run's report folder holds this run's own-score best; this attempt's file is `noop.c` at that commit in the run's agent worktree.

```c
// ================================================================================
// SECTION: BPF KERNEL CODE
// ================================================================================
// EVOLVE-BLOCK-START
// S3-FIFO: a small FIFO (~10% of the cache) admits new folios; a folio re-accessed while in
// the small queue is promoted to the main queue, otherwise evicted. The main queue is
// CLOCK-like with a frequency counter capped at 3. A ghost map remembers recently evicted
// folios so a quick re-reference is admitted straight into main.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ENOENT        2  /* include/uapi/asm-generic/errno-base.h */
#define INT64_MAX    (9223372036854775807LL)

// Set from userspace. In terms of number of pages.
// TODO: change

//#define CACHE_SIZE (((1ull << 30) * 2) / 4096)
#define CACHE_SIZE (((1ull << 20) * 200) / 4096)
const volatile size_t cache_size = 0;

struct folio_metadata {
    s64 freq;
    bool in_main;
    bool scan;
};

/* Per-thread sequential-insertion tracker used to detect the scan thread. */
struct seq_key {
    u64 ino;
    u32 tid;
    u32 pad;
};

struct seq_state {
    u64 last_idx;
    u32 run;
};

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct seq_key);
    __type(value, struct seq_state);
    __uint(max_entries, 16384);
} seq_map SEC(".maps");

/* A folio whose insertion extends a per-thread sequential run of >= SEQ_RUN is a scan folio. */
#define SEQ_RUN 3
#define SEQ_GAP 64
/* Keep at least this many scan folios resident so un-consumed readahead isn't thrown away. */
#define SCAN_MIN 512

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, 4000000);
} folio_metadata_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u32);
    //__uint(max_entries, CACHE_SIZE); // TODO: change
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

static u64 main_list;
static u64 small_list;
static u64 scan_list;
static s64 scan_list_size = 0;

/*
 * This is an approximate value based on what we choose to evict, not what is
 * actually evicted.
 */
static s64 small_list_size = 0;
static s64 main_list_size = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline struct folio_metadata *get_folio_metadata(struct folio *folio) {
    u64 key = (u64)folio;
    return bpf_map_lookup_elem(&folio_metadata_map, &key);
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline u32 folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    u32 *cnt = bpf_map_lookup_elem(&ghost_map, &key);
    if (!cnt)
        return 0;
    u32 c = *cnt;
    bpf_map_delete_elem(&ghost_map, &key);
    return c ? c : 1;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created main_list: %llu\n", main_list);

    small_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (small_list == 0) {
        bpf_printk("cache_ext: init: Failed to create small_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created small_list: %llu\n", small_list);

    scan_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (scan_list == 0) {
        bpf_printk("cache_ext: init: Failed to create scan_list\n");
        return -1;
    }

    return 0;
}

static s64 bpf_s3fifo_score_main_fn(struct cache_ext_list_node *a) {
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return INT64_MAX;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return INT64_MAX;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return INT64_MAX;
```

```
    }

    /* LFU with history: evict the sampled folio with the lowest long-term count. */
    return data->freq;
}

static int bpf_s3fifo_score_small_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Move to main list if freq > 1
    if (data->freq > 1) {
        data->in_main = true;
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Else, evict
    return CACHE_EXT_EVICT_NODE;
}

static void evict_main(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct sampling_options opts = {
        .sample_size = 32,
    };

    if (bpf_cache_ext_list_sample(memcg, main_list, bpf_s3fifo_score_main_fn, &opts,
                      eviction_ctx)) {
        bpf_printk("cache_ext: evict: Failed to sample main_list\n");
        return;
    }

    // if (__sync_sub_and_fetch(&main_list_size, eviction_ctx->nr_folios_to_evict) < 0)
    //     main_list_size = 0;
}

#define MAIN_ITER_FN(id)                                    \
static int bpf_s3fifo_score_main_iter_fn_##id(int idx, struct cache_ext_list_node *a)     \
{                                                           \
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))         \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                                            \
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))         \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                                            \
    struct folio_metadata *data = get_folio_metadata(a->folio);             \
    if (!data) {                                            \
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");         \
        return CACHE_EXT_CONTINUE_ITER;                     \
    }                                                       \
                                                            \
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);                \
    if (freq < id) {                                        \
        /*data->freq = 0;*/                                 \
        return CACHE_EXT_EVICT_NODE;                        \
    }                                                       \
                                                            \
    return CACHE_EXT_CONTINUE_ITER;                         \
}

MAIN_ITER_FN(0)
MAIN_ITER_FN(1)
MAIN_ITER_FN(2)
MAIN_ITER_FN(3)

static void evict_main_iter(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_0, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_1, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_2, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_3, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    }
}

static void evict_small(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 1, move to main list, otherwise evict.
     * (When evicting, move to tail in the meantime).
     *
     * Use the iterate interface.
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, small_list, bpf_s3fifo_score_small_fn, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate small_list\n");
        return;
    }

    if (__sync_fetch_and_sub(&small_list_size, opts.nr_folios_continue) < 0)
        small_list_size = 0;

    if (__sync_fetch_and_add(&main_list_size, opts.nr_folios_continue) < 0)
        main_list_size = opts.nr_folios_continue;
}

static int bpf_scan_evict_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    return CACHE_EXT_EVICT_NODE;
}

static void evict_scan(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, scan_list, bpf_scan_evict_fn, &opts,
                        eviction_ctx) < 0)
```

```
        bpf_printk("cache_ext: evict: Failed to iterate scan_list\n");
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    if (scan_list_size > SCAN_MIN) {
        evict_scan(eviction_ctx, memcg);
        if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
            return;
    }

    // bpf_printk("cache_ext: evict_folios: main_list_size: %lld, small_list_size: %lld, cache_size: %lld\n",
    //        main_list_size, small_list_size, cache_size);
    if (small_list_size >= cache_size / 15 || main_list_size <= 2 * small_list_size)
        evict_small(eviction_ctx, memcg);
    else
        evict_main(eviction_ctx, memcg);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        bpf_printk("cache_ext: accessed: Failed to get metadata\n");
        return;
    }

    if (data->freq < 1000000)
        __sync_fetch_and_add(&data->freq, 1);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
    u64 key = (u64)folio;
    u32 ghost_val = 1;

    // if (bpf_cache_ext_list_del(folio)) {
    //     bpf_printk("cache_ext: Failed to delete folio from sampling_list\n");
    //     return;
    // }

    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        //bpf_printk("cache_ext: evicted: Failed to get metadata\n");
        return;
    }

    if (data->scan) {
        __sync_fetch_and_sub(&scan_list_size, 1);
        bpf_map_delete_elem(&folio_metadata_map, &key);
        return;
    }

    if (data->freq > 1)
        ghost_val = data->freq > 1000000 ? 1000000 : (u32)data->freq;
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
        bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");

    if (data->in_main)
        __sync_fetch_and_sub(&main_list_size, 1);
    else
        __sync_fetch_and_sub(&small_list_size, 1);

    bpf_map_delete_elem(&folio_metadata_map, &key);

    // if (bpf_map_delete_elem(&folio_metadata_map, &key))
    //     bpf_printk("cache_ext: evicted: Failed to delete metadata\n");
}

/*
 * If folio is in the ghost map, add to tail of main list, otherwise add to tail
 * of small list.
 */
void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    u64 key = (u64)folio;
    struct folio_metadata new_meta = {
        .freq = 0,
    };

    u64 list_to_add;
    bool is_scan = false;
    u32 ghost_cnt;
    u64 idx = folio->index;
    struct seq_key sk = {
        .ino = folio->mapping->host->i_ino,
        .tid = (u32)bpf_get_current_pid_tgid(),
    };
    struct seq_state *st = bpf_map_lookup_elem(&seq_map, &sk);
    if (st) {
        if (idx > st->last_idx && idx <= st->last_idx + SEQ_GAP) {
            if (st->run < 1000000)
                st->run++;
        } else {
            st->run = 0;
        }
        st->last_idx = idx;
        is_scan = st->run >= SEQ_RUN;
    } else {
        struct seq_state ns = { .last_idx = idx, .run = 0 };
        bpf_map_update_elem(&seq_map, &sk, &ns, BPF_ANY);
    }

    if (is_scan) {
        list_to_add = scan_list;
        new_meta.scan = true;
        __sync_fetch_and_add(&scan_list_size, 1);
    } else if ((ghost_cnt = folio_in_ghost(folio))) {
        new_meta.freq = ghost_cnt;
        list_to_add = main_list;
        new_meta.in_main = true;
        __sync_fetch_and_add(&main_list_size, 1);
    } else {
        list_to_add = small_list;
        new_meta.in_main = false;
        __sync_fetch_and_add(&small_list_size, 1);
    }

    if (bpf_cache_ext_list_add_tail(list_to_add, folio)) {
        // TODO: add back to ghost_map?
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }

    if (bpf_map_update_elem(&folio_metadata_map, &key, &new_meta, BPF_ANY)) {
        // TODO: add back to ghost_map? + error check delete call?
        bpf_cache_ext_list_del(folio);
        bpf_printk("cache_ext: added: Failed to create folio metadata\n");
        return;
    }
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the S3-FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_size <size> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
        uint64_t cgroup_size;
```

```
        char *cgroup_path;
};
static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
        {"cgroup_size", 's', "SIZE", 0, "Size of the cgroup"},
        {"cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)"},
    { 0 },
};

static const uint64_t page_size = 4096;

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
        case 's':
                // TODO: move this to parse_args()
                errno = 0;
                args->cgroup_size = strtoull(arg, NULL, 10);
                if (errno)
                        args->cgroup_size = 0;

                break;
        case 'c':
                args->cgroup_path = arg;
                break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_size == 0) {
             fprintf(stderr, "Invalid cgroup size\n");
             return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set cache size in terms of number of pages. Assumes uniform page size.
    skel->rodata->cache_size = args.cgroup_size / page_size;
    fprintf(stderr, "Cgroup size: %lu bytes\n", args.cgroup_size);
    fprintf(stderr, "Cache size: %lu pages\n", skel->rodata->cache_size);

    // Resize ghost_map
    if (bpf_map__set_max_entries(skel->maps.ghost_map, skel->rodata->cache_size * 4)) {
        perror("Failed to resize ghost_map");
        ret = 1;
        goto cleanup;
    }

    // Set watch_dir
    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        ret = 1;
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        ret = 1;
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    printf("evo_policy (S3-FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;
```

```
692
693 cleanup:
694     close(cgroup_fd);
695     bpf_link__destroy(link);
696     evo_policy_bpf__destroy(skel);
697     return ret;
698 }
699 // EVOLVE-BLOCK-END
```